OptimalBinningWoE

CHEAT SHEET · SHEET 1 OF 2 · BIN, SCREEN, TRANSFORM

Optimal binning and Weight of Evidence for credit scoring and risk modeling: 37 C++ binning implementations behind one R interface, from a raw feature store to a deployed scorecard.

version 1.14.0

```
install.packages("OptimalBinningWoE")
library(OptimalBinningWoE)
```

STEP 1 — **Fit** — `obwoe()` — Bin every column, numerical and categorical, in one call.

→ **STEP 2** — **Screen** — `obwoe_select()` — One verdict and one reason per candidate. Nothing is dropped silently.

→ **STEP 3** — **Transform** — `obwoe_apply()` — Apply the fitted bins to new data, or inside a recipe.

→ **STEP 4** — **Deploy** — `obwoe_sql()` — The same transform as SQL CASE, for 14 dialects.

What the transform measures

Binning replaces a raw column by its bins; WoE replaces each bin by the log-ratio of the event and non-event distributions it holds.

$$\mathrm{WoE}_i = \ln(p_i / q_i) \quad \cdot \quad \mathrm{IV} = \sum_i (p_i - q_i) \cdot \mathrm{WoE}_i$$

p_i = share of the events that fall in bin i; q_i = share of the non-events. A positive WoE is a riskier bin.

IV STRENGTH BANDS (SIDDIQI, 2006)

Unpredictive	Weak	Medium	Strong	Suspicious
< 0.02	0.02 – 0.10	0.10 – 0.30	0.30 – 0.50	≥ 0.50

> *An IV at or above* **0.50** *is far more often leakage than signal, which is why* `obwoe_select()` *rejects it by default. Medium is where the workhorses of a scorecard sit.*

Fit

STEP 1

```
obwoe(data, target, feature = NULL,
      min_bins = 2, max_bins = 7,
      algorithm = "auto",
      control = control.obwoe())
```

`data`	Data frame holding the target and the candidates.
`target`	Column name. Binary 0/1, or multinomial 0,1,2,…
`feature`	`NULL` bins every column but the target.
`min_bins` `max_bins`	Target range for the number of bins.
`algorithm`	One of the 28 names, or `"auto"`: `jedi` for a binary target, `jedi_mwoe` for a multinomial one. A fixed rule, not a search for the highest IV.
`control`	See `control.obwoe()` on sheet 2.

WHAT COMES BACK: AN `OBWOE` OBJECT

```
m <- obwoe(german, target = "default", max_bins = 6)

m$summary      # feature, type, algorithm, n_bins, total_iv
m$results$age  # bin, woe, iv, count, count_pos, count_neg,
               #   cutpoints, converged, iterations
m$target_type  # "binary" or "multinomial"
```

LOOK AT IT

`summary(m)`	Feature table, `sort_by = "iv"`, decreasing.
`print(m)`	Compact fit report.
`plot(m, type=)`	`"iv"` ranking bar chart · `"woe"` profile of one feature · `"bins"` counts and event rate. Pass `feature=`, `top_n = 15`.

Screen

STEP 2

Two criteria govern admission: Information Value strength and guaranteed rank ordering. Every candidate returns a row, so an automatic verdict stays reviewable.

```
sel <- obwoe_select(m,
    detail = "summary",     # or "full": one row per bin
    iv_min = 0.02, iv_max = 0.5,
    require_monotonic = "numeric",  # "all" | "none"
    monotonicity = "weak",  # or "strict"
    min_bins = 2, max_bins = Inf,
    min_bin_pct = 0, allow_degenerate = FALSE,
    top_n = NULL, sort_by = "iv", decreasing = TRUE)
```

COLUMNS THAT CARRY THE VERDICT

`total_iv, iv_class`	IV and its Siddiqi band.
`ks, gini, auc`	Discrimination of the binned score.
`monotonic` `monotonic_strict`	Event-rate ordering, plus `monotonic_direction`, `n_violations`, `spearman`.
`n_degenerate_bins`	Bins with no events or no non-events.
`quality`	Excellent · Good · Fair · Rejected.
`selected`	The flag. `reason` and `reason_desc` say why.

REASON CODES (JOINED BY ";")

`OK` `IV_BELOW_MIN` `IV_SUSPICIOUS` `NOT_MONOTONIC` `TOO_FEW_BINS` `TOO_MANY_BINS` `SMALL_BIN` `DEGENERATE_BIN` `NOT_IN_TOP_N` `BINNING_ERROR`

```
keep <- sel$feature[sel$selected]
```

Transform

STEP 3

```
obwoe_apply(data, obj,
            suffix_bin = "_bin", suffix_woe = "_woe",
            keep_original = TRUE, na_woe = 0)
```

Adds `<feature>_bin` and `<feature>_woe` per binned column. `na_woe` is the fallback for an unseen category or an unmodeled missing value. Numerical intervals are half-open on the right, `(a, b]`.

THE MANUAL PATH, ONE COLUMN AT A TIME

`ob_cutpoints_num()`	Bin a numeric vector at cut points you supply.
`ob_cutpoints_cat()`	Same for a categorical vector.
`ob_apply_woe_num()`	Map a numeric vector through a fitted result; `missing_values = c(-999)`.
`ob_apply_woe_cat()`	Categorical counterpart; `missing_values = c("NA","Missing","")`.

Evaluate

```
g <- obwoe_gains(m, feature = "duration")
g <- obwoe_gains(scored, target = "default",
                 feature = "score_decile",
                 use_column = "direct", n_groups = 10)
```

`use_column`	`"auto"` · `"bin"` · `"woe"` · `"direct"`.
`sort_by`	`"id"` (the algorithm's own order, default) · `"woe"` · `"event_rate"` · `"bin"`.
`n_groups`	Cut a continuous score into that many quantile groups.

18 STATISTICS PER BIN

`count` `count_pct` `pos_count` `neg_count` `pos_rate` `neg_rate` `pos_pct` `neg_pct` `odds` `log_odds` `woe` `iv` `cum_pos_pct` `cum_neg_pct` `ks` `lift` `capture_rate`

```
g$metrics   # ks, gini, auc, total_iv, ks_bin
plot(g, type = "cumulative")  # "ks" | "lift" | "woe_iv"
```

> *Related helpers:* `obwoe_gains_score()` *for the bin-level engine behind a fitted result,* `obwoe_gains_variable()` *for any binned data frame plus a grouping column.*

Prepare the raw column

```
ob_preprocess(feature, target,
   num_miss_value = -999, char_miss_value = "N/A",
   outlier_method = "iqr",   # "zscore" | "grubbs"
   outlier_process = FALSE,
   preprocess = "both",
   iqr_k = 1.5, zscore_threshold = 3,
   grubbs_alpha = 0.05)
```

Missing values become a value of their own rather than a dropped row, so the bin that holds them carries its own WoE and stays visible in the model document.

```
ob_check_distincts(x, target)  # cardinality and
                               # separation warnings
```

Runnable from a clean session

The Statlog (German Credit) benchmark ships with the package.

```
german <- read.csv(gzfile(system.file(
  "extdata", "germancredit.csv.gz",
  package = "OptimalBinningWoE")),
  stringsAsFactors = FALSE)

german$default <- 1L - german$credit_risk
german$credit_risk <- NULL

m   <- obwoe(german, "default", max_bins = 6)
sel <- obwoe_select(m)
out <- obwoe_apply(german, m)
```

Choose, score, deploy

CHEAT SHEET · SHEET 2 OF 2 · ALGORITHMS, SCORECARD, SQL

28 algorithm names cover 37 implementations — 21 accept a numerical feature, 16 a categorical one, 9 both.
The last column says what the code does, which is not always what the name suggests.

C++ via Rcpp

```
obwoe_algorithms()
vignette("algorithms")
```

The 28 algorithms

N UMERICAL · C ATEGORICAL

FAMILY	NAME	N	C	WHAT THE CODE DOES
Greedy merging under an IV or WoE criterion	`jedi`	N	C	IV-driven merging, monotone WoE *(default)*
	`jedi_mwoe`	N	C	JEDI for a multinomial target; one-vs-rest WoE
	`mob`	N	C	Greedy merging that enforces monotone WoE
	`mrblp`	N	·	No linear program, no likelihood-ratio pre-bins
	`mblp`	N	·	Greedy merging; no linear program is solved
	`oslp`	N	·	Greedy merging; no linear program is solved
	`bb`	N	·	Merges the lowest-IV bin; no search tree
	`gmb`	·	C	Merges the adjacent pair that keeps most IV
	`mba`	·	C	Merges by rate similarity, then least IV loss
	`milp`	·	C	Greedy merging; no integer program is solved
	`swb`	·	C	Greedy Jensen–Shannon merging; no window
	`udt`	N	C	Entropy-ranked cuts (N), JS merging (C)
Exact dynamic programming	`ivb`	·	C	Exact DP over rate-sorted pre-bins
	`dp`	N	C	Same DP (C); greedy merging (N)
	`sblp`	·	C	Same DP; rare levels pooled by event rate
Recursive entropy partitioning	`fast_mdlp`	N	·	Fayyad–Irani criterion; monotone by default
Bottom-up MDL cost	`mdlp`	N	·	Merges by MDL cost; not Fayyad–Irani
Statistical merging	`cm`	N	C	ChiMerge: smallest-χ^2 merges; α stops C only
	`fetb`	N	C	Merges the highest Fisher p-value pair; no α
Isotonic regression	`ir`	N	·	PAVA pools adjacent violating bins
Density-based	`ldb`	N	·	Gaussian-KDE minima; quantiles fill to min_bins
	`lpdb`	N	·	KDE extrema and inflections; no polynomial
Distribution-based initialization	`ewb`	N	·	Equal-width start, then supervised merging
	`kmb`	N	·	Evenly spaced centroids; no k-means step
	`ubsd`	N	·	Mean ± 1–2 SD, equal-width edges, then merging
Divergence-driven	`dmiv`	N	C	Nine divergence options; WoE defaults to log-odds
Streaming	`sketch`	N	C	Sketch-based cuts (N), heavy levels (C); in memory
Metaheuristic	`sab`	·	C	Simulated annealing over level assignments

Picking one

- **The default:** `jedi` (`jedi_mwoe` if multinomial); `"auto"` is a fixed rule, not a search.
- **Exact DP over rate-sorted pre-bins:** `ivb`, `sblp`, categorical `dp`.
- **A stopping rule can lower the bin count:** `fast_mdlp`, categorical `cm`.
- **Monotone event rates by construction:** `ir`.
- **Very large n or many levels:** `sketch`; the data must still fit in memory.

Points and scaling

```
sc <- obwoe_scale(pdo = 20, score_ref = 600,
                  odds_ref = 50,
                  direction = "higher_is_safer")
s  <- obwoe_score(link, sc, round = TRUE)
```

$$\text{Score} = \text{Offset} - \text{Factor} \cdot \eta, \quad \text{Factor} = \text{PDO} / \ln 2$$

The models predict the log-odds of the *event*, so the sign is negative: a high WoE is a risky bin and must score *fewer* points. Two identities pin the scale — a case at `odds_ref` scores exactly `score_ref`, and doubling the good-to-bad odds adds exactly `pdo` points, everywhere.

Tune the binning

```
control.obwoe(
   bin_cutoff = 0.05,            # min share per bin
   max_n_prebins = 20,           # pre-merge ceiling
   convergence_threshold = 1e-6,
   max_iterations = 1000,
   bin_separator = "%;%",        # joins merged levels
   verbose = FALSE)
```

max_n_prebins is a modeling decision, not a detail. *Pre-binning runs before the algorithm merges anything, so a heavy tail can be smeared into one quantile cell and lost with no warning. Leave the default, and tune it against held-out IV for long-tailed continuous predictors only.*

Call one algorithm directly

```
r <- ob_numerical_ir(german$duration, y,
        min_bins = 3, max_bins = 5,
        bin_cutoff = 0.05, max_n_prebins = 20,
        auto_monotonicity = TRUE)

r <- ob_categorical_jedi(german$purpose, y,
        bin_separator = "%;%")
```

Every name in the table is exported as `ob_numerical_<name>()`, `ob_categorical_<name>()`, or both. They return the same bin, woe, iv and count vectors that `obwoe()` stores per feature.

The whole pipeline in one call

```
card <- obwoe_scorecard(german,
    target = "default", split = 0.7,
    validation = NULL,      # an out-of-time frame
    exclude = NULL, feature = NULL,
    binning = list(max_bins = 6),
    screening = list(iv_max = 0.5),
    engine = "glm",         # "obwoe" | "glmnet" | list()
    control = control.obwoe_scorecard(),
    file = "scorecard.xlsx", seed = 42)
```

Split, bin, screen by IV and correlation, fit, scale to points, and write the model document. Three properties are enforced rather than assumed:

- Binning is fitted on the **training rows only**, so the hold-out is not inflated by supervised leakage.
- A variable whose WoE coefficient comes out **negative** is dropped and the model refitted — the WoE already carries the direction.
- The generated points SQL reproduces the R card score **exactly**, unseen categories included.

READING THE OBJECT

`card$points`	The fixed points table, one row per variable and bin.
`card$screening`	`obwoe_select()` plus a `stage` column: `in_model`, `sign_rejected`, `corr_pruned`, `constant_woe`, `screened_out`.
`card$samples`	Per sample: scores, gains table, headline metrics.
`card$stability`	PSI for the score and for every variable in the model.
`card$warnings`	Everything the pipeline wants a human to read.

```
predict(card, new_data, type = "score")
# "card" | "link" | "prob" | "woe"
```

Redundancy and stability

```
obcorr(df, method = "all", threads = 0)
```

`pearson` `spearman` `kendall` `hoeffding` `distance` `biweight` `pbend` `robust` `alternative`

```
p <- obwoe_prune(woe_df, ranking = keep,
                 cutoff = 0.7, method = "pearson")
p$keep; p$dropped   # variable, correlated_with, correlation
```

Prune in the **WoE space** — that is the space the model actually sees.

```
s <- obwoe_psi(base, compare, n_groups = 10)
s$psi; s$table; s$flag
```

PSI	FLAG	READING
< 0.10	stable	No action
0.10 – 0.25	watch	Monitor the shift
> 0.25	act	Population has moved

A band populated in one vintage and empty in the other reports `Inf` rather than being smoothed away — a segment that has vanished is exactly what monitoring exists to catch.

tidymodels

```
rec <- recipe(default ~ ., data = german) |>
  step_obwoe(all_predictors(), outcome = "default",
             algorithm = "auto",
             min_bins = 2, max_bins = 10,
             bin_cutoff = 0.05, output = "woe",
             na_woe = 0)
```

`prep()`, `bake()`, `tidy()` and `required_pkgs()` are all implemented. Four parameters are tunable, each with a `dials` constructor:

TUNE()	DIALS	RANGE
`algorithm`	`obwoe_algorithm()`	28 names
`min_bins`	`obwoe_min_bins()`	2 – 5
`max_bins`	`obwoe_max_bins()`	5 – 20
`bin_cutoff`	`obwoe_bin_cutoff()`	0.01 – 0.10

Deploy to SQL

STEP 4

```
obwoe_sql(m, table = "risk.applications",
   features = keep,
   output = "woe",        # "bin" | "index" | "both"
   style = "select",      # "case" | "cte" | "view"
   dialect = "postgres",
   na_value = 0, explicit_bounds = TRUE,
   quote_identifiers = "auto", file = NULL)
```

`ansi` `postgres` `mysql` `mariadb` `sqlserver` `oracle` `spark` `hive` `databricks` `bigquery` `snowflake` `redshift` `duckdb` `sqlite`

```
CASE WHEN duration IS NULL THEN 0
     WHEN duration <= 7 THEN -1.31218638896617
     WHEN duration > 7 AND duration <= 10 THEN -0.4519851
     ... ELSE 0 END AS duration_woe
```

Every expression opens with an explicit `IS NULL` branch, because `NULL <= 5` is `NULL` in SQL, not `FALSE`.

obwoe_report(card, file) writes the 13-sheet model document: summary, scorecard, coefficients, bin statistics, screening funnel, correlations before and after pruning, score gains, PSI, cut-off strategy, SQL in WoE and in points form, reproducibility.